

Plataformas e integrações: Use o Claude Code no VS Code — Curso Hands-on

Platforms and Integrations: Use Claude Code in VS Code — Curso Hands-on

Ao final deste curso, você terá instalado, configurado e usado o Claude Code para melhorar um trecho de código real — com as suas próprias mãos.

Antes de Arregaçar as Mangas

Este curso é mão na massa do começo ao fim. Não tem enrolação, não tem slides cheios de bullet points e não tem lição de casa chata. Cada oficina apresenta um problema real, você resolve, e — pronto — tem uma vitória concreta no bolso. A teoria aparece só quando é necessária para o próximo passo, e não um segundo antes.

O que você precisa ter aberto agora:

- VS Code instalado no seu computador (versão 1.95 ou mais recente)
- Uma conta gratuita na Anthropic — crie em claude.ai
- Acesso ao console da Anthropic em console.anthropic.com (para gerar sua chave de API)
- Conexão com a internet
- Um arquivo de código qualquer salvo na sua máquina — pode ser Python, JavaScript, ou até um script de um exercício antigo

 **Regra de ouro do curso:** Errar é parte do processo. Se algo não funcionar na primeira tentativa, vá direto para a seção "Se travar" daquela oficina — ela existe exatamente para isso. Código que roda de primeira suspeita de ter sido copiado do gabarito.

O Mapa da Mão na Massa

Oficina	O problema que você vai resolver	Tempo
1	Instalar o Claude Code e confirmar que ele está funcionando	20 min
2	Fazer o Claude entender o seu código e explicá-lo	20 min
3	Pedir ao Claude para melhorar um trecho — e revisar antes de aceitar	20 min
4	Usar o Claude para encontrar e corrigir um erro real	20 min
Projeto Final	Refatorar um módulo completo do zero ao documentado	20 min

Cada oficina entrega uma vitória pequena e real. Faça em sequência — cada uma usa o que você aprendeu na anterior.

Oficina 1 — Instalar e Ligar o Motor

O Problema

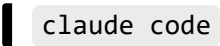
Você ouviu falar no Claude Code, abriu o VS Code, olhou para a tela... e nada. Nenhum ícone novo, nenhum painel mágico. É porque a extensão ainda não está instalada. Nesta oficina você vai instalar, configurar e trocar a primeira mensagem com o assistente — tudo em menos de 20 minutos.

O que você vai usar

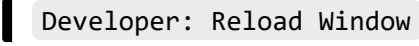
- VS Code aberto na sua tela
- Conta na Anthropic (precisa ter criado em console.anthropic.com)
- Conexão com a internet

Mão na massa

Parte 1 — Instalar a extensão

1. Abra o VS Code.
2. Pressione `Ctrl + Shift + X` (Windows/Linux) ou `Cmd + Shift + X` (Mac). O painel de extensões vai aparecer na lateral esquerda.
3. No campo de busca no topo do painel, digite:

4. Procure a extensão publicada por **Anthropic Inc.** (o nome do publicador aparece em cinza logo abaixo do nome da extensão).

 **Captura de tela sugerida:** painel de extensões com o resultado da busca, destacando o nome "Anthropic Inc." abaixo do título da extensão.

1. Clique no botão **Install** (em azul).
2. Aguarde entre 30 e 60 segundos até o botão mudar para **Uninstall** — isso significa que a instalação terminou.
3. Se aparecer um botão **Reload Required** ou uma notificação pedindo para recarregar a janela, clique nela. Se não aparecer, pressione `Ctrl + Shift + P` (ou `Cmd + Shift + P`) e digite:
 Pressione Enter.

Parte 2 — Gerar sua chave de API

A chave de API é como uma senha que o VS Code usa para se identificar para o Claude. Você vai gerar uma agora.

8. Abra o navegador e acesse console.anthropic.com. Faça login com a sua conta Anthropic.
9. No menu lateral esquerdo, clique em **API Keys**.

 *Captura de tela sugerida: tela do console Anthropic mostrando o menu lateral com "API Keys" destacado.*

10. Clique no botão **Create API Key**.
11. No campo de nome, escreva algo descritivo. Por exemplo:

12. Clique em **Create Key**.

13. Uma sequência de letras e números vai aparecer. Ela começa com `sk-ant-`. **Copie agora** — ela só aparece uma vez. Se fechar sem copiar, vai precisar criar outra.

⚠ **Guarde bem essa chave.** Nunca cole ela em um arquivo que você vai enviar por e-mail, colocar no Git ou compartilhar. Ela é o acesso à sua conta — tratar como senha.

Parte 3 — Conectar o VS Code ao Claude

14. Volte ao VS Code. Olhe na barra lateral esquerda — deve ter aparecido um ícone novo. Geralmente é um ícone de raio (⚡) ou um ícone com o logo da Anthropic. Clique nele.

15. Se uma caixa de diálogo pedir sua chave de API, cole a chave que você copiou no passo 13 e pressione Enter.

16. Se não aparecer a caixa automaticamente, pressione `Ctrl + Shift + P` (ou `Cmd + Shift + P`) e busque:

`Claude Code: Sign In` Selecione a opção e cole a chave quando pedido.

Parte 4 — Primeiro contato

17. Clique no ícone do Claude Code na barra lateral para abrir o painel de chat.

18. No campo de texto no fundo do painel, escreva:

`Olá! Quanto é 7 vezes 8?`

19. Pressione Enter ou clique no botão de envio.

✅ Deu certo?

Se o Claude respondeu "56" (ou algo parecido, com alguma explicação simpática), parabéns — a extensão está instalada, autenticada e funcionando. O motor está ligado.

🛠 Se travar

Problema	O que fazer
Não aparece nenhum ícone do Claude na barra lateral após instalar	Feche e reabra o VS Code completamente (não só recarregue a janela)
Aparece erro "Invalid API Key" ou "Authentication failed"	Volte ao console.anthropic.com , gere uma chave nova e repita o passo 15
A busca por "claude code" no marketplace não retorna nada	Verifique se a sua versão do VS Code é 1.95 ou superior em <i>Help</i> → <i>About</i>

🚀 Quer ir além?

- Gere uma segunda chave de API com o nome "Backup" e guarde em local seguro.
- Explore o painel de configurações do Claude Code (clique na engrenagem dentro do painel) e veja quais opções estão disponíveis.
- Pesquise no marketplace do VS Code outras extensões da Anthropic e veja o que elas fazem.

Oficina 2 — Faça o Claude Ler Seu Código

O Problema

Você tem um arquivo de código na tela. Pode ser algo que você escreveu, algo que você herdou de outra pessoa, ou um exercício de aula que nunca ficou muito claro. A questão é: você quer entender o que aquele código faz — e rápido. Nesta oficina você vai aprender a formular perguntas que realmente funcionam, usando o arquivo aberto como contexto automático.

O que você vai usar

- VS Code com Claude Code instalado (Oficina 1 concluída)
- Um arquivo de código aberto no editor — use o seu próprio ou crie o arquivo de exemplo abaixo

Mão na massa

Parte 1 — Preparar o arquivo de exemplo

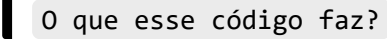
Se você não tem um arquivo à mão, crie um agora:

1. No VS Code, pressione `Ctrl + N` (ou `Cmd + N`) para abrir um arquivo novo.
2. Pressione `Ctrl + Shift + P` e busque `Change Language Mode`. Selecione **Python**.
3. Cole o código abaixo no arquivo:

```
def processar_notas(notas):
    total = 0
    for n in notas:
        total = total + n
    media = total / len(notas)
    aprovados = []
    for n in notas:
        if n >= 6:
            aprovados.append(n)
    return media, len(aprovados)
```

4. Salve o arquivo como `notas.py` em qualquer pasta (`Ctrl + S` ou `Cmd + S`).

Parte 2 — O prompt errado (para você ver a diferença)

5. Abra o painel do Claude Code clicando no ícone na barra lateral.
6. No campo de texto, escreva exatamente isso:

7. Envie e leia a resposta.

Perceba: o Claude pode responder de forma genérica ou pode pedir mais detalhes. Isso acontece porque a pergunta não deu contexto suficiente.

 **Dica:** O Claude Code vê automaticamente o conteúdo do arquivo que está aberto e em foco no editor. Mas se você não mencionar o arquivo na pergunta, ele pode não saber exatamente para onde olhar — especialmente se houver vários arquivos abertos.

Parte 3 — O prompt que funciona

8. No campo de texto, escreva agora:

```
Tenho o arquivo notas.py aberto. Ele contém uma função chamada processar_notas(). Explique o que ela faz, passo a passo, como se eu nunca tivesse visto Python na vida.
```

9. Envie e compare com a resposta anterior.

 *Captura de tela sugerida: o painel do Claude Code mostrando a resposta detalhada, com o arquivo `notas.py` visível no editor ao fundo.*

Parte 4 — Perguntas de aprofundamento

10. Agora pergunte:

```
Essa função tem algum problema se eu passar uma lista vazia para ela? O que acontece?
```

11. Após ler a resposta, pergunte:

```
Como eu poderia corrigir esse problema? Não mexa no arquivo ainda — só me mostre como ficaria o código corrigido.
```

Guarde a sugestão na memória — você vai usá-la na Oficina 4.

✅ Deu certo?

O Claude explicou a função `processar_notas()` com clareza, identificou o risco da lista vazia e sugeriu uma correção — sem tocar no arquivo. Se conseguiu isso, você dominou a habilidade mais importante do curso: perguntar bem.

Se travar

Problema	O que fazer
O Claude diz que não vê nenhum arquivo	Clique uma vez no código dentro do editor para "focar" nele, depois volte ao painel e tente de novo
A resposta veio em inglês	Adicione ao final do seu prompt: "Responda em português."
O Claude modificou o arquivo sem você pedir	Clique em "Undo" (<code>Ctrl + Z</code>) para desfazer e reenvie o prompt com a instrução "não mexa no arquivo ainda"

Quer ir além?

- Abra dois arquivos ao mesmo tempo e use a sintaxe `@nomedoarquivo` no prompt para especificar qual deles o Claude deve analisar.
 - Peça ao Claude para identificar quais partes do código são mais difíceis de entender e por quê.
 - Selecione apenas duas linhas do código com o mouse e pergunte: "O que esse trecho faz?"
-

Oficina 3 — Melhore o Código (com Você no Controle)

O Problema

Você tem um código que funciona, mas está feio. Variáveis com nomes sem sentido, funções que fazem três coisas ao mesmo tempo, zero comentários. Você sabe que dá para melhorar, mas não sabe exatamente como. E tem medo de mexer e quebrar tudo. Nesta oficina você vai aprender o ciclo seguro de refatoração: o Claude sugere, você revisa e você decide.

O que você vai usar

- VS Code com Claude Code instalado
- O arquivo `notas.py` da Oficina 2 (ou o código de exemplo abaixo, se pulou a anterior)

Mão na massa

Parte 0 — Garantir que o arquivo está pronto

Se você não tem o `notas.py` da Oficina 2, crie-o agora com este conteúdo:

```
def processar_notas(notas):
    total = 0
    for n in notas:
        total = total + n
    media = total / len(notas)
    aprovados = []
    for n in notas:
        if n >= 6:
            aprovados.append(n)
    return media, len(aprovados)
```

Parte 1 — Pedir análise, não modificação

O segredo aqui é um protocolo de três etapas: Analisar → Planejar → Executar. Nunca pule direto para o "Executar".

1. Abra `notas.py` no editor. Abra o painel do Claude Code.
2. Envie este prompt:

```
Analise a função processar_notas() no arquivo notas.py. Liste os problemas de legibilidade e organização que você encontrar. Não mexa no código ainda.
```

3. Leia a lista de problemas que o Claude apresentar. Ele provavelmente vai mencionar nomes de variáveis pouco descritivos (`n`, `total`) e a ausência de type hints e docstrings.

✳️ **Não esqueça:** "Type hints" são anotações que dizem o tipo de dado esperado em cada variável (por exemplo, `notas: list[float]`). "Docstring" é um comentário especial que explica o que a função faz. Você vai ver os dois em ação daqui a pouco.

Parte 2 — Pedir o plano

4. Agora envie:

Mostre-me o plano de mudanças que você faria, passo a passo. Liste cada alteração e explique por que ela melhora o código. Ainda não aplique nada.

5. Leia o plano. Concorde com tudo? Tem alguma mudança que você não quer? Este é o momento de falar.

6. Se quiser vetar algo, diga ao Claude. Por exemplo:

Concordo com tudo, exceto a mudança X. Prossiga sem ela.

Parte 3 — Executar e revisar o diff

7. Quando estiver satisfeito com o plano, envie:

Agora aplique as mudanças. Antes de confirmar, mostre-me o diff.

O "diff" é a comparação visual entre o código antigo e o novo. Linhas em vermelho vão embora. Linhas em verde chegam.

 *Captura de tela sugerida: o diff viewer do VS Code mostrando linhas removidas em vermelho (com sinal "-") e linhas adicionadas em verde (com sinal "+"), com os botões Accept e Reject visíveis.*

8. Leia o diff com calma. Você entende cada mudança?

9. Se estiver tudo bem, clique em **Accept**. Se não gostar de algo, clique em **Reject** e diga o que quer diferente.

Parte 4 — Conferir o resultado

10. Depois de aceitar, seu arquivo deve estar parecido com este (os detalhes podem variar):

```
def processar_notas(notas: list[float]) -> tuple[float, int]:
    """
    Calcula a média e conta os aprovados de uma lista de notas.

    Args:
        notas: Lista de notas numéricas.

    Returns:
        Uma tupla com (média, número de aprovados).
    """
    if not notas:
        return 0.0, 0

    media = sum(notas) / len(notas)
    aprovados = sum(1 for n in notas if n >= 6)

    return media, aprovados
```

11. Salve o arquivo (**Ctrl + S** ou **Cmd + S**).

✓ Deu certo?

O arquivo `notas.py` agora tem a função com nome de variáveis mais claros, type hints e uma docstring explicando o que ela faz. Se você conseguiu aceitar as mudanças depois de revisar o diff — sem um ataque de pânico — você está pronto para a próxima fase.

 **Curiosidade técnica (pode pular):** O `sum(1 for n in notas if n >= 6)` que o Claude provavelmente sugeriu é uma "generator expression" — uma forma compacta de contar itens que

atendem a uma condição, sem criar uma lista temporária na memória. É mais eficiente, mas equivalente ao loop original.

 Se travar

Problema	O que fazer
O Claude aplicou a mudança sem mostrar o diff	Use <code>Ctrl + Z</code> para desfazer, e desta vez especifique no prompt: "mostre o diff antes de confirmar, aguarde minha aprovação"
Não sei se devo aceitar ou rejeitar uma mudança	Pergunte ao Claude: "Explique essa linha específica em linguagem simples antes de eu aceitar"
O arquivo ficou com erro de sintaxe após aceitar	Veja qual linha está marcada em vermelho, descreva o erro ao Claude e peça que corrija

 Quer ir além?

- Peça ao Claude para reescrever a mesma função de duas formas diferentes e explique qual prefere.
 - Adicione uma segunda função ao arquivo (por exemplo, uma que filtre apenas as notas reprovadas) e peça ao Claude para documentá-la também.
 - Pergunte ao Claude quais testes unitários ele criaria para a função — sem precisar aplicá-los agora.
-

Oficina 4 — Caça ao Bug com Copiloto

O Problema

Todo código tem um bug esperando para aparecer no pior momento possível. Nesta oficina você vai introduzir um erro proposital em um código, ver ele explodir, e então usar o Claude como copiloto de depuração — descrevendo o problema e deixando o assistente ajudar a rastrear a causa. Depurar com IA é uma habilidade que salva horas.

O que você vai usar

- VS Code com Claude Code instalado
- O arquivo `notas.py` da Oficina 3 (melhorado e documentado)
- Terminal integrado do VS Code

Mão na massa

Parte 1 — Introduzir o bug

Vamos plantar um erro clássico: usar o operador errado numa comparação.

1. Abra `notas.py` no VS Code.
2. Encontre a linha que define os aprovados. Ela deve estar parecida com:

```
aprovados = sum(1 for n in notas if n >= 6)
```

3. Troque `>=` por `>` (apenas remova o `=`):

```
aprovados = sum(1 for n in notas if n > 6)
```

4. Salve o arquivo.

Parte 2 — Rodar o código e observar o comportamento errado

5. Abra o terminal integrado do VS Code: pressione `Ctrl + `` (a crase, geralmente na tecla abaixo do Esc).
6. No terminal, crie um arquivo de teste rápido. Digite:

```
python3 -c "from notas import processar_notas; print(processar_notas([5, 6, 7, 8]))"
```

(No Windows, pode ser `python` em vez de `python3`.)

7. Pressione Enter. O resultado esperado seria `(6.5, 3)` — média 6,5 e 3 aprovados (notas 6, 7 e 8). Mas o código bugado vai retornar `(6.5, 2)` — deixando a nota 6 de fora.

 *Captura de tela sugerida: terminal do VS Code mostrando o comando executado e o resultado incorreto.*

Parte 3 — Descrever o bug para o Claude

- 8. Abra o painel do Claude Code.
- 9. Envie este prompt (preencha com o resultado real que você viu):

Tenho o arquivo `notas.py` aberto. A função `processar_notas()` está retornando um resultado errado. Passei a lista `[5, 6, 7, 8]` e esperava o resultado `(6.5, 3)` – média 6,5 e 3 aprovados (notas 6, 7 e 8). Mas recebi `(6.5, 2)`. Analise o código e aponte onde está o erro. Não corrija ainda.

- 10. Leia o diagnóstico do Claude. Ele deve identificar que a condição `n > 6` exclui a nota exatamente igual a 6, que deveria ser considerada aprovada.

Parte 4 — Corrigir com revisão

- 11. Após confirmar que o diagnóstico faz sentido para você, envie:

Corrija apenas a linha com a condição de aprovação. Mostre o diff antes de aplicar.

- 12. Revise o diff. A única mudança deve ser `> 6` virando `>= 6`.
- 13. Aceite a correção.

Parte 5 — Testar novamente

- 14. No terminal, rode o mesmo comando de teste do passo 6.
- 15. Desta vez o resultado deve ser `(6.5, 3)`. Bug eliminado.

 **Dica:** Quando for descrever um bug para o Claude, sempre inclua: (1) o que você esperava receber, (2) o que recebeu de verdade, e (3) o input que usou no teste. Com esses três elementos, o diagnóstico é muito mais rápido.

 Deu certo?

O terminal mostrou `(6.5, 3)` após a correção. Você usou o Claude para diagnosticar um bug sem deixar ele simplesmente "resolver tudo" — você entendeu o problema antes de aceitar a correção. Isso é depuração assistida de verdade.

 Se travar

Problema	O que fazer
O terminal diz <code>ModuleNotFoundError</code>	Verifique se você está na mesma pasta do arquivo <code>notas.py</code> . Digite <code>ls</code> (Mac/Linux) ou <code>dir</code> (Windows) no terminal para conferir. Use <code>cd nomeDaPasta</code> para navegar.
O Claude não identificou o bug	Seja mais direto: "A condição de aprovação na linha X usa o operador errado. Qual deveria ser?"
O diff mostrou mudanças em mais linhas do que esperado	Clique em Reject, peça ao Claude para corrigir apenas a linha da condição de aprovação e nada mais.

 Quer ir além?

- Introduza um segundo bug proposital (por exemplo, divida `total` por `len(notas) - 1`) e repita o processo de diagnóstico.
 - Peça ao Claude para criar um teste automatizado que detectaria o bug que você corrigiu, caso ele reaparecesse no futuro.
 - Pergunte ao Claude: "Que outros bugs potenciais existem nessa função?" — e veja o que ele encontra.
-

Projeto Final — Módulo Refatorado do Zero ao Documentado

A missão

Você recebeu um módulo legado. O código funciona (mais ou menos), mas os nomes das funções são incompreensíveis, não tem uma linha de comentário, os bugs estão à espreita e não existe nenhum documento explicando o que aquilo faz. Sua missão é usar tudo que você praticou nas quatro oficinas para transformar esse módulo numa versão limpa, documentada e testável — usando o Claude Code como copiloto em cada etapa.

Mão na massa

1. No VS Code, crie um novo arquivo chamado `turma.py` e cole o código abaixo:

```
def proc(d):
    r = []
    for x in d:
        nm = x['n']
        ns = x['notas']
        t = 0
        for i in ns:
            t = t + i
        m = t / len(ns)
        ap = 0
        for i in ns:
            if i > 6:
                ap = ap + 1
        r.append({'nome': nm, 'media': m, 'aprovados': ap})
    return r

def rel(d):
    for x in d:
        print(x['nome'] + ': media=' + str(x['media']) + ' aprovados=' + str(x['aprovados']))
```

2. Salve o arquivo.

3. Abra o painel do Claude Code e envie:

```
Tenho o arquivo turma.py aberto. Ele contém duas funções com nomes e variáveis pouco legíveis. Explique o que cada função faz, em linguagem simples, sem modificar nada.
```

4. Leia a explicação. Certifique-se de que entendeu o que `proc()` e `rel()` fazem antes de continuar.

5. Envie:

```
Agora liste todos os problemas de qualidade que você encontra: nomes ruins, bugs, ausência de documentação, riscos com entradas inválidas. Não corrija ainda.
```

6. Leia a lista completa. Anote mentalmente (ou no papel) quais problemas fazem sentido para você.

7. Envie:

Proponha um plano de refatoração completo: nomes novos para as funções e variáveis, adição de type hints, tratamento de lista vazia e docstrings. Liste o plano antes de aplicar.

8. Revise o plano. Se concordar, envie:

Aplique o plano completo. Mostre o diff antes de confirmar.

9. Analise o diff com cuidado. Certifique-se de que você entende cada mudança antes de clicar em Accept.

10. Após aceitar, envie:

Gere um cabeçalho de módulo para o arquivo turma.py com: descrição do que o módulo faz, exemplo de uso das duas funções e uma nota sobre o que fazer se a lista de entrada estiver vazia.

11. Revise o cabeçalho sugerido, ajuste se quiser, e cole manualmente no topo do arquivo (ou peça ao Claude para inserir).

12. Salve o arquivo.

13. No terminal integrado, rode um teste rápido:

```
python3 -c "  
from turma import processar_turma  
dados = [  
    {'n': 'Ana', 'notas': [8, 7, 9]},  
    {'n': 'Bruno', 'notas': [5, 4, 6]}  
]  
print(processar_turma(dados))  
"
```

(Substitua `processar_turma` pelo nome que o Claude sugeriu para a função `proc()`.)

14. Confirme que o resultado faz sentido: Ana deve ter média 8.0 e Bruno deve ter média 5.0.

❏ Como saber que terminou

- ☐ O arquivo `turma.py` não contém mais nenhuma variável de nome único (`x`, `r`, `d`, `i`, `t`, `m`, `ap`)
- ☐ As duas funções têm nomes que descrevem o que elas fazem
- ☐ Há type hints nos parâmetros e no retorno de pelo menos uma das funções
- ☐ Há uma docstring em pelo menos uma das funções
- ☐ O código trata o caso de lista vazia sem travar
- ☐ O cabeçalho do módulo está presente no topo do arquivo
- ☐ O teste do passo 13 rodou sem erros e retornou valores coerentes
- ☐ Você consegue explicar, com suas próprias palavras, o que cada função faz

Se você marcou todos os oito itens: missão cumprida.

A Parte dos Dez — Prompts que Funcionam de Verdade

Dez instruções rápidas que você pode usar agora mesmo com o Claude Code. Guarde esta seção nos favoritos.

1. Para entender um código desconhecido:

Explique o que essa função faz, passo a passo, como se eu não soubesse programar.

2. Para melhorar a legibilidade sem mudar a lógica:

Renomeie as variáveis e funções para nomes mais descritivos. Não mude a lógica, apenas os nomes. Mostre o diff antes de aplicar.

3. Para adicionar documentação:

Adicione docstrings no estilo Google a todas as funções deste arquivo. Mostre o resultado antes de aplicar.

4. Para detectar bugs em potencial:

Quais são os cenários de entrada que poderiam fazer essa função falhar? Liste sem modificar nada.

5. Para depurar um erro com mensagem de erro:

Recebi este erro ao rodar o código: [cole a mensagem aqui]. O arquivo que está aberto é [nome]. O que está errado e como corrigir?

6. Para pedir sugestão sem autorizar mudança:

Sugira como melhorar essa função, mas não mexa no arquivo. Só me mostra como ficaria.

7. Para entender um diff antes de aceitar:

Explique em linguagem simples o que cada parte do diff está mudando e por quê.

8. Para trabalhar com múltiplos arquivos:

Estou com @arquivo1.py e @arquivo2.py abertos. Como a função X de @arquivo1.py se conecta com a função Y de @arquivo2.py?

9. Para gerar um resumo do projeto:

Com base nos arquivos abertos, escreva um parágrafo descrevendo o que este projeto faz, para incluir em um README.

10. Para pedir explicação de um conceito técnico que apareceu na resposta:

Você usou o termo "generator expression". Explique o que é em uma frase, com um exemplo simples.

Conseguiu! E agora?

Você instalou, configurou, perguntou, refatorou, caçou um bug e entregou um módulo transformado. Isso não é pouca coisa — a maioria das pessoas para no "instalar e ver o que acontece". Você foi além: aprendeu o ciclo de revisão, entendeu que o Claude é um copiloto e não um piloto automático, e percebeu que perguntar bem é tão importante quanto qualquer habilidade técnica.

Agora o próximo passo é simples: use o Claude Code no próximo arquivo real que você precisar trabalhar. Não espere o projeto perfeito. Pegue aquele script empoeirado que você nunca entendeu direito, aquele documento que precisa de revisão, aquele código de exercício que nunca ficou claro — e repita o ciclo: explica, planeja, revisa, aceita. Com o tempo, essa sequência vai virar instinto. E aí você vai olhar para trás e estranhar como alguém conseguia trabalhar sem isso.